CALIDAD EN PROYECTOS DE TI

Pedro Javier de León Coy

Carné: 7690-22-8894

Universidad Mariano Gálvez de Guatemala

Seminario de Tecnologías de la Información

pdeleonc5@miumg.edu.gt

Resumen

Este artículo explica qué aspectos ayudan a mantener la calidad de un proyecto de tecnologías de la información desde que se plantea la idea hasta que se entrega. El objetivo es identificar acciones sencillas que permitan evitar errores y obtener un resultado útil para las personas que utilizarán el sistema.

Para desarrollar el tema se revisaron recomendaciones sobre calidad del software, seguridad y trabajo en equipo, y se relacionaron con un ejemplo de una aplicación para consultar notas universitarias.

A partir de esa revisión se encontró que la calidad no depende solamente de que un programa funcione: también importa comprender las necesidades del usuario, comprobar las funciones durante el desarrollo, proteger la información y atender los problemas que van apareciendo.

Otro punto importante es mantener la comunicación entre quienes participan en el proyecto, porque muchos errores surgen cuando no todos tienen claro lo que deben hacer.

En conclusión, revisar el trabajo desde el principio resulta más útil que esperar hasta la entrega para descubrir fallas. Un proyecto de calidad es aquel que cumple su propósito y puede utilizarse con confianza.

Palabras clave: calidad del software, planificación, pruebas, seguridad, usuarios.

Desarrollo del tema

Cuando se trabaja en un proyecto de tecnología, es fácil concentrarse en terminar las funciones y dejar las revisiones para el final. El problema es que un sistema puede estar completo y aun así tener dificultades importantes. Por ejemplo, puede guardar datos de forma incorrecta, tardar demasiado en responder o resultar confuso para las personas que lo utilizan. Por eso, hablar de calidad no es solamente hablar de un programa sin errores. También es preguntarse si realmente resuelve el problema para el que fue creado.

El primer paso debería ser comprender bien lo que se necesita. Antes de escribir código conviene conversar con quienes van a utilizar el sistema, conocer qué tareas realizan y cuáles son sus principales dificultades. Si esto no se hace, el equipo puede dedicar varias semanas a construir algo que al final no es lo que se esperaba. Una lista clara de requisitos, aunque sea sencilla, ayuda a que todos trabajen con la misma idea y permite comprobar después si se cumplió lo solicitado.

También hace falta organizar el trabajo. Es importante definir qué se hará primero, quién será responsable de cada actividad y cuánto tiempo se tiene disponible. No significa que todo deba salir exactamente como se planeó, porque en cualquier proyecto pueden aparecer cambios. Lo importante es que esos cambios se conversen y no se hagan sin pensar en las consecuencias. La guía de Scrum,

por ejemplo, destaca la importancia de revisar los avances y adaptar el trabajo cuando se encuentra algo que puede mejorarse (Schwaber y Sutherland, 2020).

Una de las acciones que más ayudan a cuidar la calidad es probar el sistema mientras se desarrolla. A veces una función parece estar bien porque funciona en la computadora de quien la programó, pero falla cuando otra persona intenta utilizarla. Revisar formularios, búsquedas, permisos y mensajes de error permite detectar situaciones que podrían pasar desapercibidas. También conviene probar casos menos comunes, como ingresar información incompleta o utilizar el sistema desde otro dispositivo. No se trata de hacer pruebas solamente para cumplir un requisito, sino de descubrir qué necesita corregirse.

Para evaluar un producto también existen referencias que ayudan a no olvidar aspectos importantes. La norma ISO/IEC 25010:2023 presenta un modelo de calidad para productos de software, que sirve para revisar características como el funcionamiento, la seguridad y la facilidad de uso (International Organization for Standardization, 2023). No es necesario memorizar toda la norma para aplicar su idea principal: un sistema debe evaluarse desde varios puntos de vista y no únicamente por la cantidad de funciones que tiene.

La seguridad merece atención especial. Si una aplicación guarda contraseñas, notas o datos personales, esa información debe estar protegida. Por ejemplo, no sería correcto que cualquier estudiante pudiera entrar a la cuenta de otro o cambiar una calificación. Por eso conviene revisar los accesos, cuidar las contraseñas y evitar que se muestre información privada a quien no corresponde. El Marco de Desarrollo de Software Seguro del NIST recomienda incorporar estas prácticas durante el desarrollo y no dejarlas como una tarea de último momento (Souppaya et al., 2022).

Otro detalle que suele pasarse por alto es la comunicación. Si una persona modifica una parte del sistema y no se lo comenta a los demás, pueden aparecer errores que antes no existían. Mantener un registro de los cambios, compartir avances y avisar cuando surge un problema ayuda a evitar confusiones. Además, cuando alguien del equipo encuentra una falla, lo más útil es buscar cómo resolverla en lugar de esperar a que otro la descubra. La calidad es una responsabilidad compartida, incluso cuando cada integrante tiene tareas diferentes.

Pensemos en una aplicación para consultar notas universitarias. Al inicio podría parecer suficiente que el docente ingrese las calificaciones y que el estudiante las vea en pantalla. Sin embargo, todavía habría que comprobar que las notas se guarden correctamente, que cada persona consulte únicamente sus propios datos y que el sistema siga funcionando cuando muchos estudiantes ingresen al mismo tiempo. También sería útil pedir a algunos docentes y estudiantes que prueben la aplicación y expliquen qué les resulta difícil. Sus comentarios podrían revelar problemas que el equipo no había visto.

Algo que ayudaría bastante es acordar desde el inicio cómo se sabrá que una función está lista. Por ejemplo, si se crea un formulario para registrar estudiantes, no basta con que permita escribir un nombre: también debería avisar cuando falta información, evitar registros repetidos y mostrar un mensaje claro al guardar. Tener estos criterios por escrito facilita que otra persona compruebe el resultado sin depender de explicaciones del programador. Incluso una pequeña lista de verificación al terminar cada actividad puede marcar la diferencia.

Así se puede avanzar con más seguridad, porque no se pasa a la siguiente parte dejando pendientes que después son más difíciles de encontrar. Esto no exige herramientas costosas, sino constancia y un poco de orden.

Por último, entregar el proyecto no significa que ya no habrá nada que mejorar. Con el uso pueden aparecer errores nuevos, dudas o necesidades diferentes. Por eso es importante documentar lo realizado y dejar una forma de reportar problemas. En mi opinión, un buen resultado no es el que promete ser perfecto, sino el que se ha revisado con cuidado y puede mejorarse sin poner en riesgo la información ni el trabajo de los usuarios.

Observaciones y comentarios

Al revisar el tema, me llamó la atención que varios problemas de calidad pueden evitarse con acciones bastante sencillas, como preguntar al usuario, probar a tiempo y comunicar los cambios. En un proyecto universitario es común dar prioridad a terminar rápido, pero eso puede provocar que los errores se acumulen. Una mejora práctica sería reservar tiempo para revisar cada avance antes de continuar con el siguiente.

Conclusiones

1. La calidad empieza cuando se entiende bien lo que necesita el usuario y se organiza el trabajo de forma clara.
2. Las pruebas y la seguridad deben revisarse durante el desarrollo, no solamente cuando el sistema está terminado.
3. Escuchar comentarios y corregir errores ayuda a entregar una solución más útil y confiable.

Bibliografía

International Organization for Standardization. (2023). *ISO/IEC 25010:2023: Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model*. `https://www.iso.org/standard/78176.html`

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. `https://scrumguides.org/scrum-guide.html`

Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST SP 800-218). National Institute of Standards and Technology. `https://doi.org/10.6028/NIST.SP.800-218`